

RAI-818: INTELLIGENT **MOBILE ROBOTICS**

- **Configuration Space**

Text : Principles of Robot Motion – Theory, Algorithms and Implementations
H. Choset, K.M. Lynch, S. Hutchinson, G. Kantor, W. Burgard, et al.

INSTRUCTOR:
DR. KUNWAR FARAZ AHMED KHAN

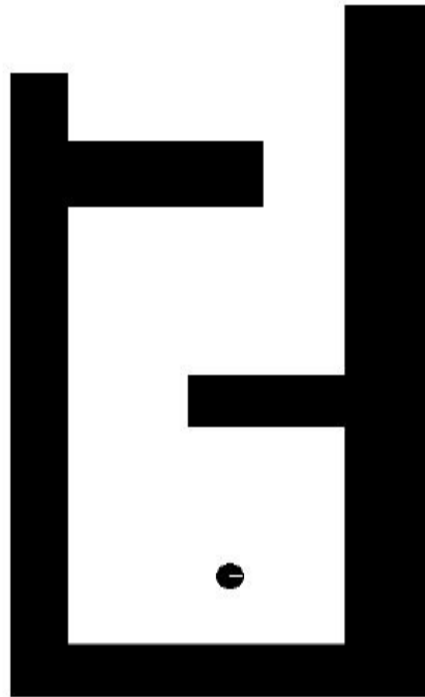
Configuration Space



- C- Space
- Non-Point Robots

What if the Robot is not a Point

- Until now we have only thought of a robot as a point moving in free space



Most mobile robots should not be modelled as a point



Nor should robots with extended linkages that may contact obstacles



Defining futures

Configuration Space



- C- Space
- Non-Point Robots

What if the Robot is not a Point

- To create realistic motion plans we should be able to accurately specify the position of the robot which is not a point.
- We must be able to give a specification of the location of all points on a robot since we need to ensure that no point on the robot collides with an obstacle
- On the other hand it is much more easier to deal with a point moving in the workspace
- Therefore we reach a compromise wherein we expand the obstacles and in order to reduce the robot to a point



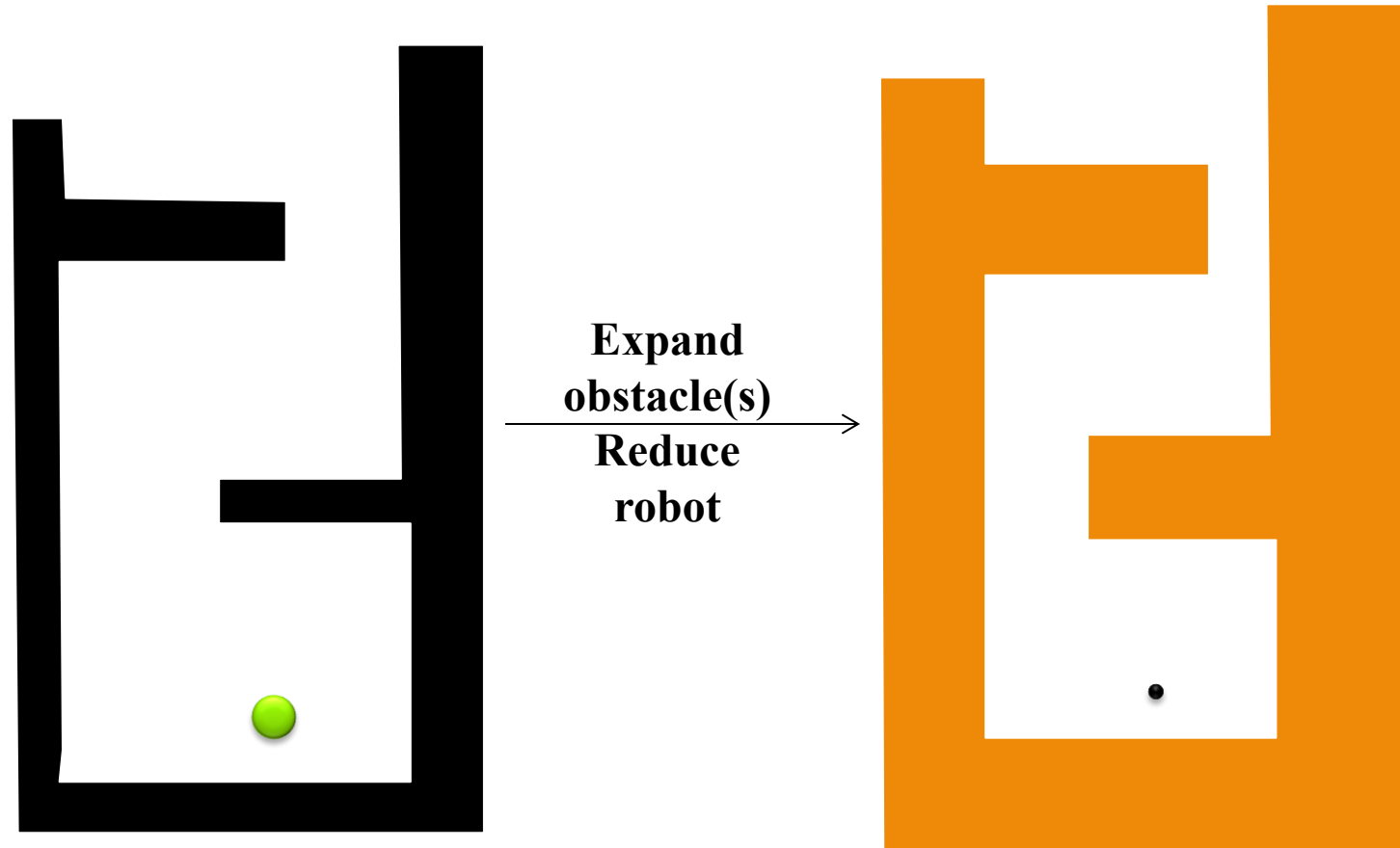
Defining futures

Configuration Space



- C- Space
- Non-Point Robots

What if the Robot is not a Point



Defining futures

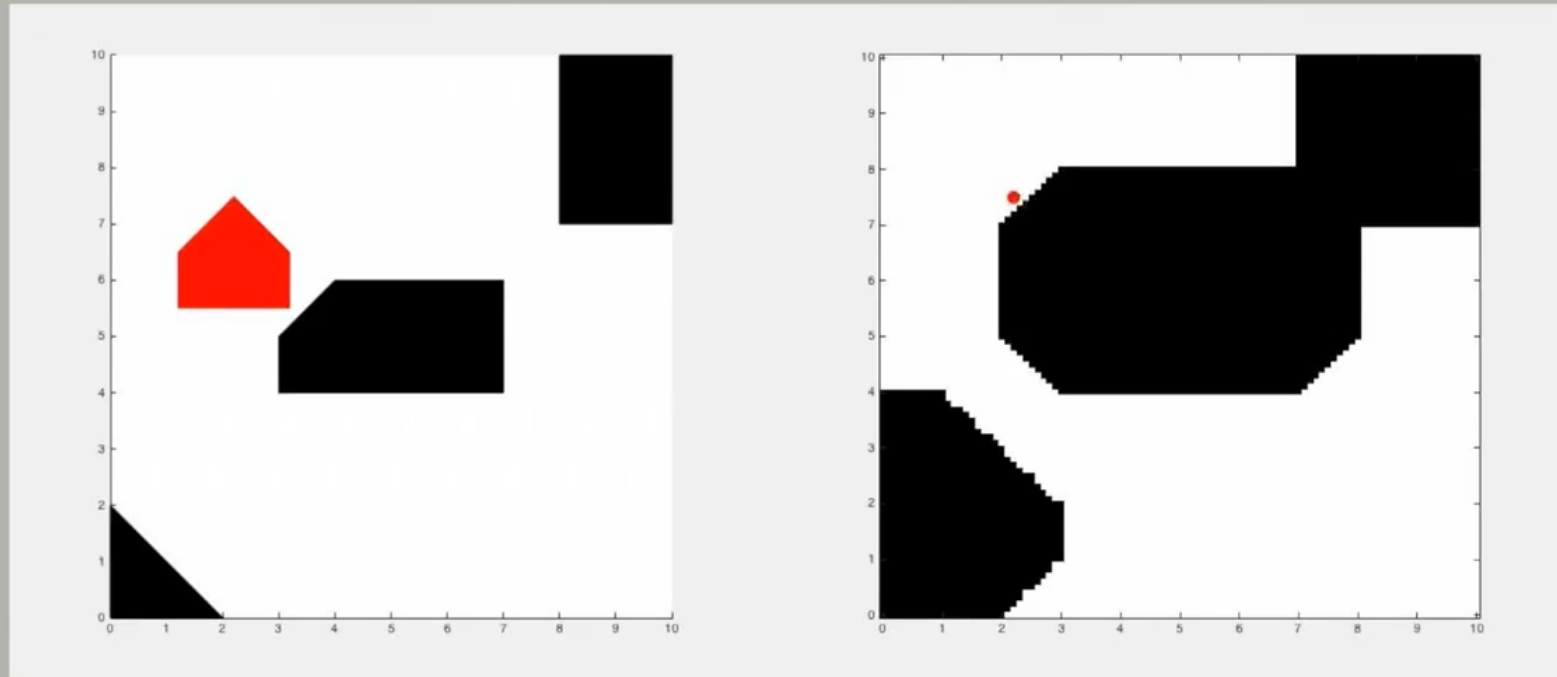
Configuration Space



- C- Space
- Non-Point Robots

What if the Robot is not a Point

Configuration Space Obstacles



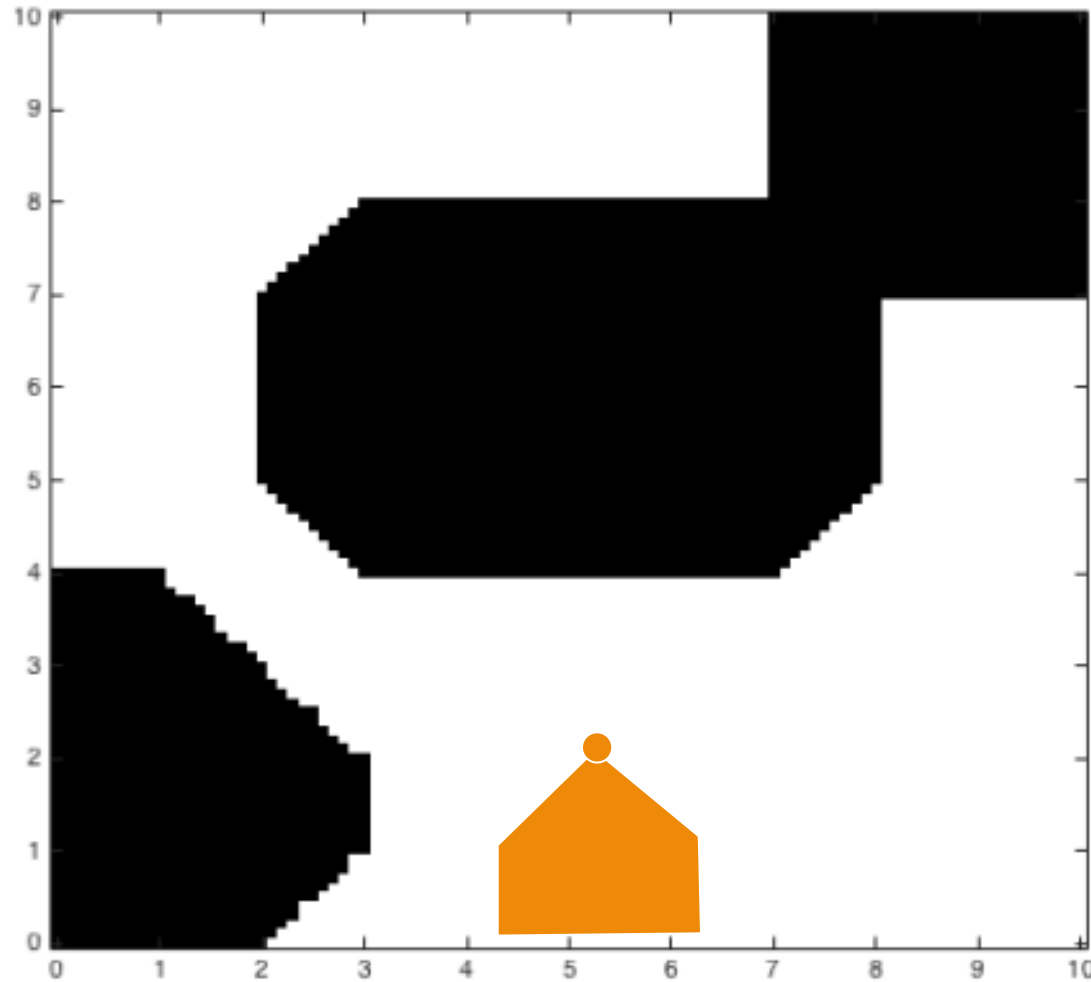
Defining futures

Configuration Space



- C- Space
- Non-Point Robots

What if the Robot is not a Point



Defining futures

Configuration Space

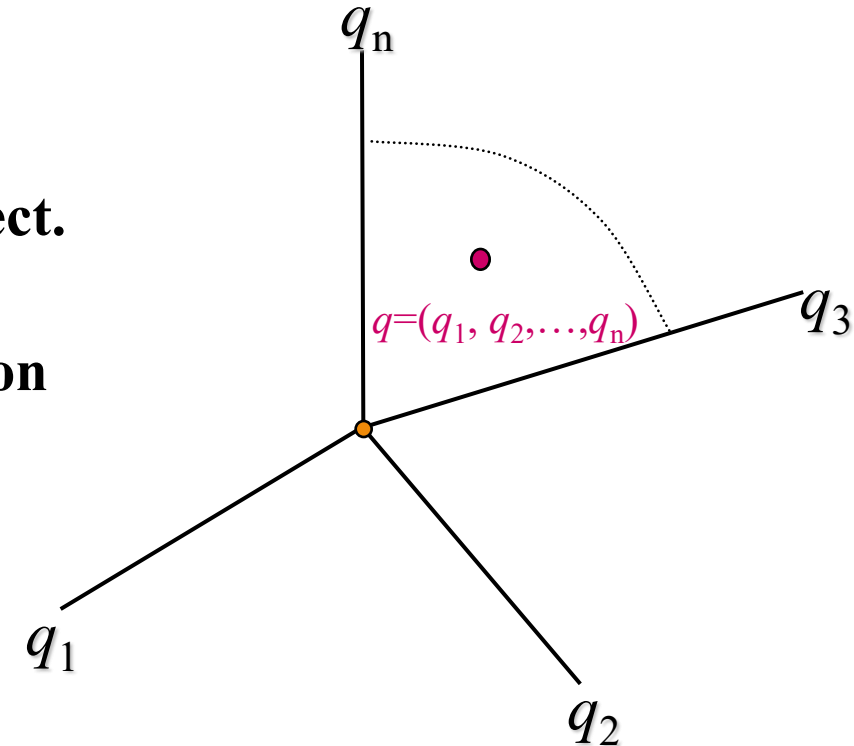


- C- Space
 - Non-Point Robots
 - What's a C-Space

What is a Configuration Space

- The *configuration* q of a moving object is a specification of the position of *every* point on the object.
 - Usually a configuration is expressed as a vector of position & orientation parameters:

$$q = (q_1, q_2, \dots, q_n)$$



- The *configuration space* or *C-space* Q is the set of all possible configurations.
 - A *configuration* is a point in C



Configuration Space



- C- Space
 - Non-Point Robots
 - What's a C-Space

The Dimensions of a Configuration Space

- The *dimension of a configuration space* is the *minimum* number of parameters needed to specify the configuration of the object completely.
- It is also called the *number of degrees of freedom (dofs)* of a moving object.
- Robots moves in a two- or three-dimensional Euclidean space
 - Ambient space or *workspace W*

[Sometimes, the workspace has a different meaning: e.g., the *workspace of the robot arm = the set of point reachable by the end effector*]



Defining futures

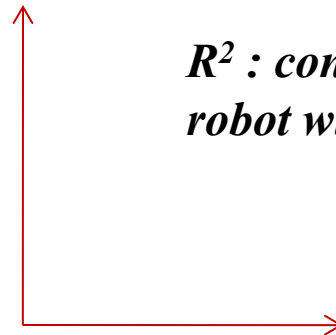
Configuration Space



- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples

Example of a Configuration Space

- A circular robot that can translate without rotating in the plane (R^2).
 - If we know the *radius* r of the robot, then by specifying the centre $q = (x, y)$ of the robot, we completely specify the all the points in the robot, i.e., given $q = (x, y)$, the set of points occupied by the robot is
$$R(q) = R(x, y) = \{(x', y') \in R^2 \mid (x - x')^2 + (y - y')^2 \leq r^2\},$$
 - So, a configuration of the planar circular robot is $q = (x, y)$, and the configuration space is two-dimensional R^2
 - Here workspace and the configuration space are both R^2 .



R^2 : configuration space of circular planar robot without rotation



Defining futures

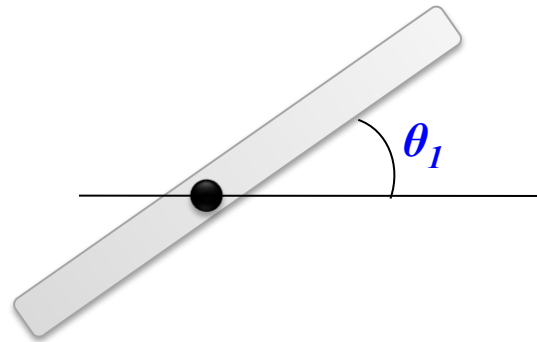
Configuration Space

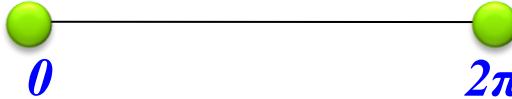


- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples

Example of a Configuration Space

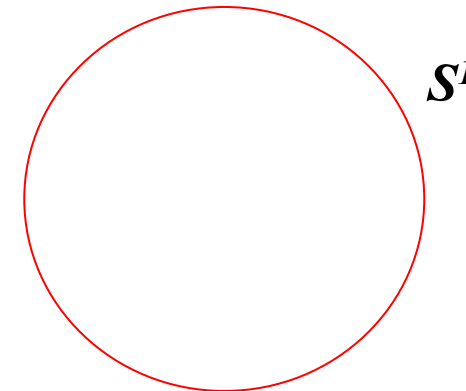
Configuration of a rotating bar fixed at a point



$$q = \theta_1$$
$$Q = \{\theta \mid 0 \leq \theta < 2\pi\}$$


$\longleftrightarrow R^1$

- Actually, $Q \cong S^1$
 $S^1 = \{x \in R^2 \mid \|x\| = 1\}$
 $S^1 \neq R^1$
Both are one-dimensional
but have different
topologies



Defining futures

Configuration Space



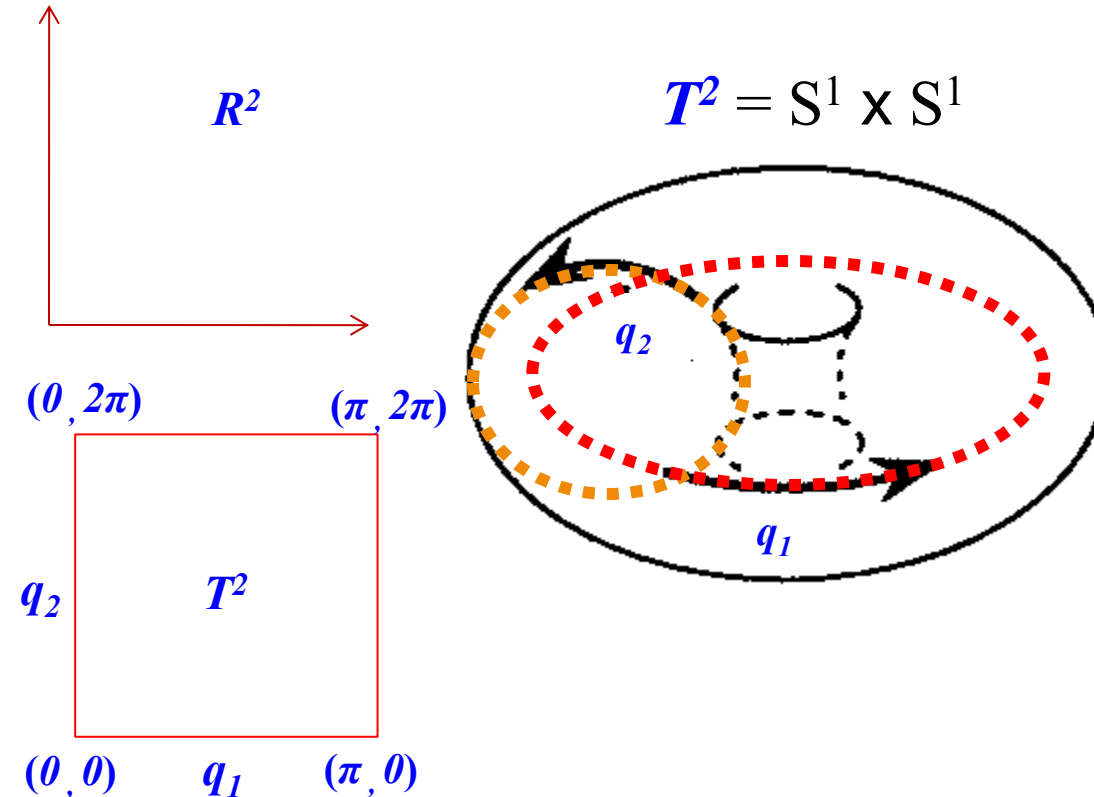
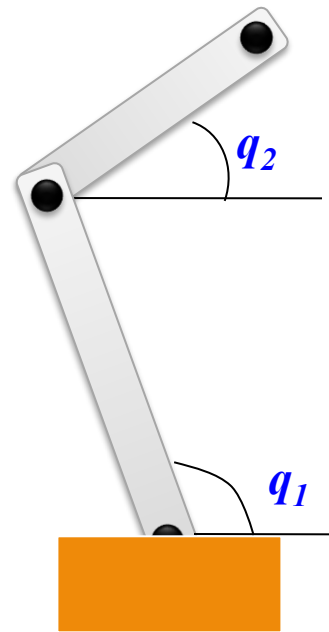
- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples

Example of a Configuration Space

What is the configuration space of two-joint planar robot arm?

$$q = (q_1, q_2)$$

Two-dimensional, but is not $\mathbb{R}^2$, but $T^2 = S^1 \times S^1$



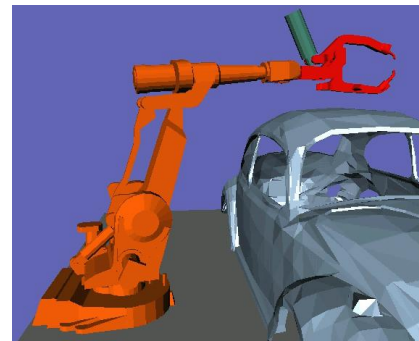
Defining futures

Configuration Space

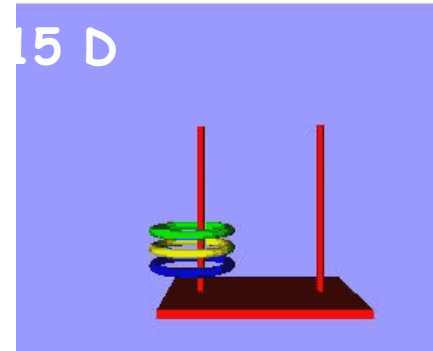


- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples

Every Robot Maps to a Point in its C-Space . . .



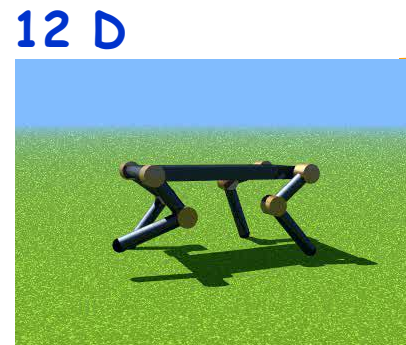
6 D



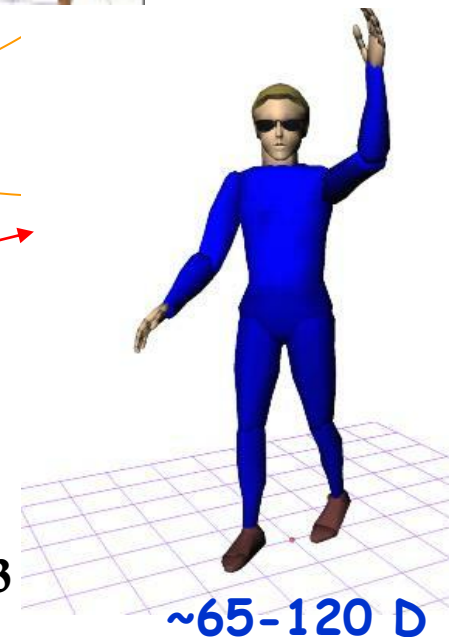
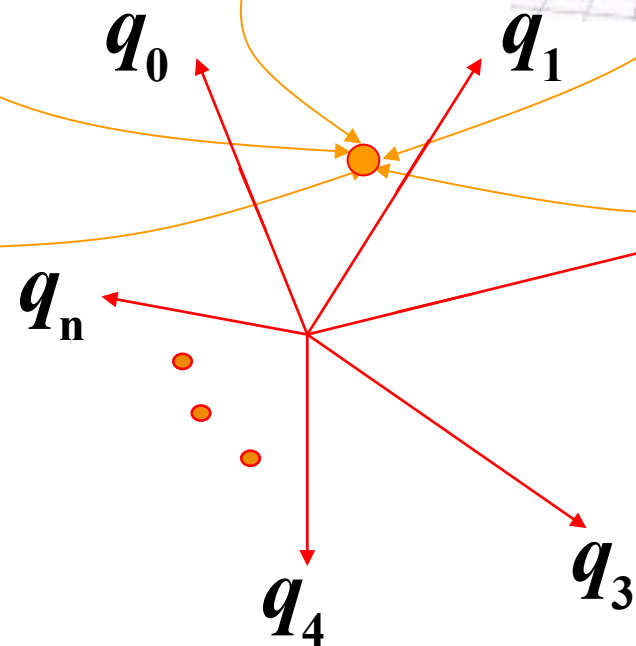
15 D



~40 D



12 D



~65-120 D



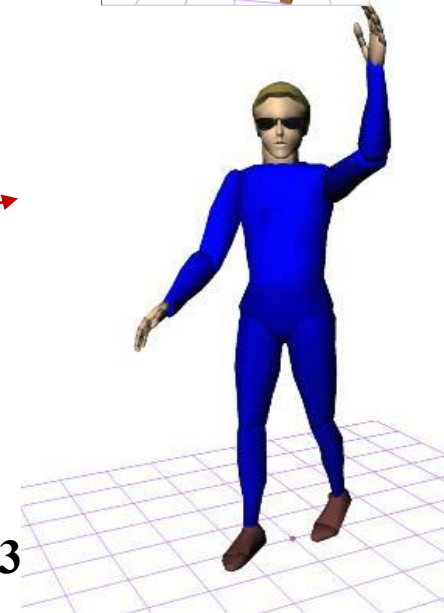
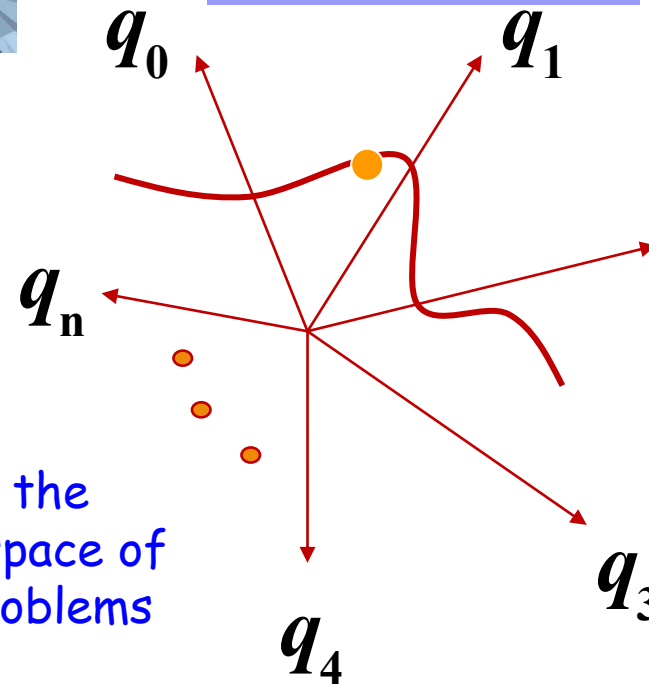
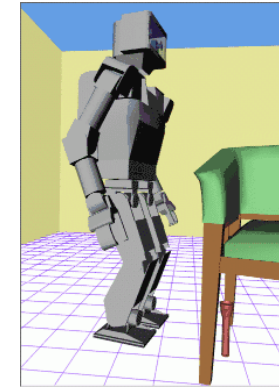
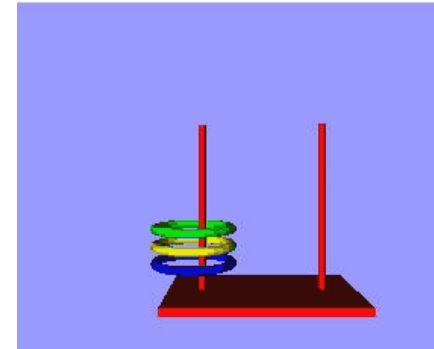
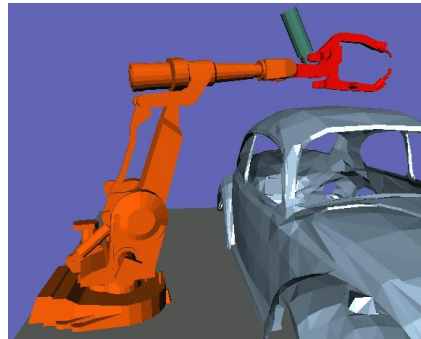
Defining futures

Configuration Space



- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples

... and every robot path is a curve in C-Space.



So, the C-space is the continuous state space of motion planning problems



Defining futures



Configuration Space

- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples
 - Kinematics

What are this Arm's Forward Kinematics
(How does its position depend on its joint angles?)

Find (x, y) in terms of α and β ...

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} L_1 c_\alpha \\ L_1 s_\alpha \end{bmatrix} + \begin{bmatrix} L_2 c_+ \\ L_2 s_+ \end{bmatrix} \quad \text{position}$$

What are this arm's inverse kinematics?

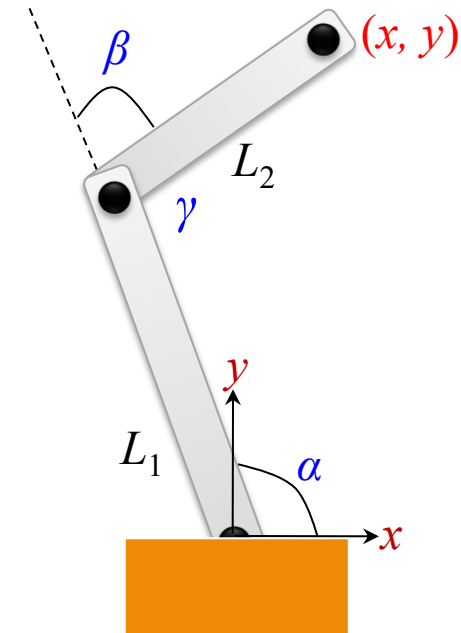
finding joint angles from Cartesian coordinates
- via a geometric or algebraic approach

Given (x, y) and L_1 and L_2 , what are the values of α, β, γ ?

$$\gamma = \cos^{-1} \left(\frac{x^2 + y^2 - L_1^2 + L_2^2}{2L_1 L_2} \right)$$

$$\beta = 180 - \gamma$$

$$\alpha = \sin^{-1} \left(\frac{L_2 \sin(\gamma)}{x^2 + y^2} \right) + \tan^{-1} \left(\frac{y}{x} \right)$$



Keeping it Simple

$$\begin{aligned} c_\alpha &= \cos(\alpha), & s_\alpha &= \sin(\alpha) \\ c_\beta &= \cos(\beta), & s_\beta &= \sin(\beta) \\ c_+ &= \cos(\alpha + \beta), & s_+ &= \sin(\alpha + \beta) \end{aligned}$$



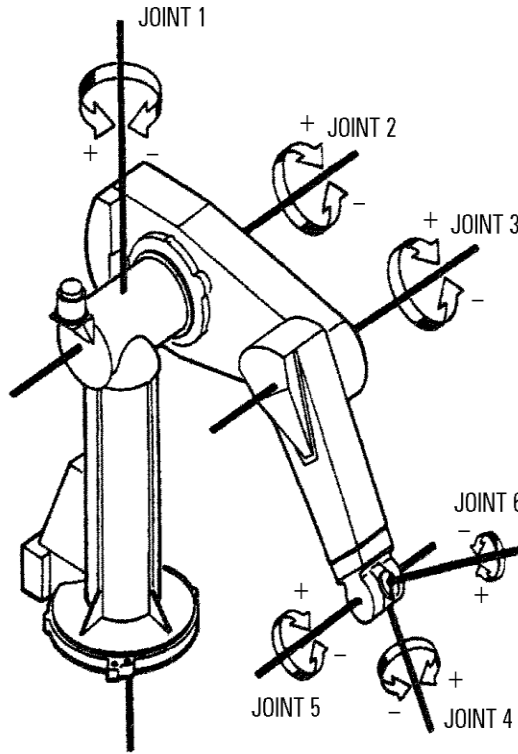
Defining futures

Configuration Space



- C- Space
- Non-Point Robots
- What's a C-Space
- Examples
- Kinematics

Puma's Inverse Kinematics



```
% Solve for theta(1)
r=sqrt(Px^2 + Py^2);
if (n1 == 1),
    theta(1)= atan2(Py,Px) + asin(d3/r);
else
    theta(1)= atan2(Py,Px) + pi - asin(d3/r);
end
```

```
% Solve for theta(2)
V114= Px*cos(theta(1)) + Py*sin(theta(1));
r=sqrt(V114^2 + Pz^2);
Psi = acos((a2^2-d4^2-a3^2+V114^2+Pz^2)/
    (2.0*a2*r));
theta(2) = atan2(Pz,V114) + n2*Psi;
```

```
% Solve for theta(3)
num = cos(theta(2))*V114+sin(theta(2))*Pz-a2;
den = cos(theta(2))*Pz - sin(theta(2))*V114;
theta(3) = atan2(a3,d4) - atan2(num, den);
```

```
V113 = cos(theta(1))*Ax + sin(theta(1))*Ay;
V323 = cos(theta(1))*Ay - sin(theta(1))*Ax;
V313 = cos(theta(2)+theta(3))*V113 +
    sin(theta(2)+theta(3))*Az;
theta(4) = atan2((n4*V323),(n4*V313));
```

```
% Solve for theta(5)
num = -cos(theta(4))*V313 - V323*sin(theta(4));
den = -V113*sin(theta(2)+theta(3)) +
    Az*cos(theta(2)+theta(3));
theta(5) = atan2(num,den);
```

```
% Solve for theta(6)
V112 = cos(theta(1))*Ox + sin(theta(1))*Oy;
V132 = sin(theta(1))*Ox - cos(theta(1))*Oy;
V312 = V112*cos(theta(2)+theta(3)) +
    Oz*sin(theta(2)+theta(3));
V332 = -V112*sin(theta(2)+theta(3)) +
    Oz*cos(theta(2)+theta(3));
V412 = V312*cos(theta(4)) - V132*sin(theta(4));
V432 = V312*sin(theta(4)) + V132*cos(theta(4));
num = -V412*cos(theta(5)) - V332*sin(theta(5));
den = - V432;
theta(6) = atan2(num,den);
```



Defining futures

Configuration Space



- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples

Other Examples and Food for Thought

- A rotating bar that translates along the rotation axis
 - what is its C-space?
 - what is its workspace
- A two-link manipulator
 - what is its C-space?
 - what is its workspace?
 - Suppose there are joint limits, does this change the C-space?
 - The workspace?



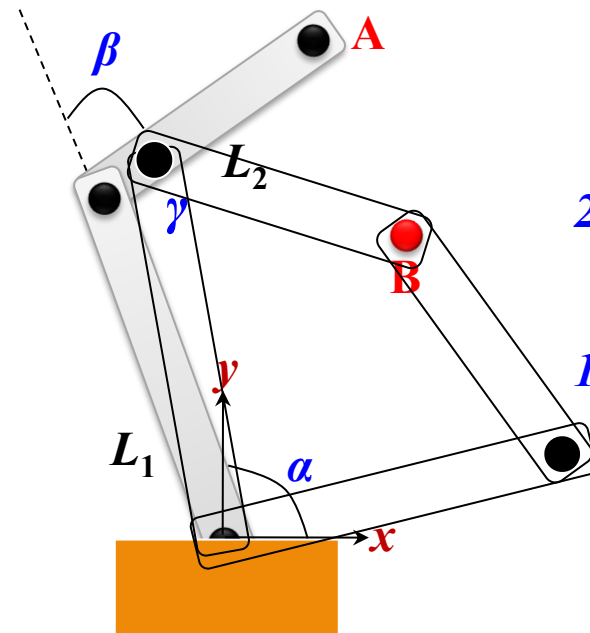
Defining futures



Configuration Space

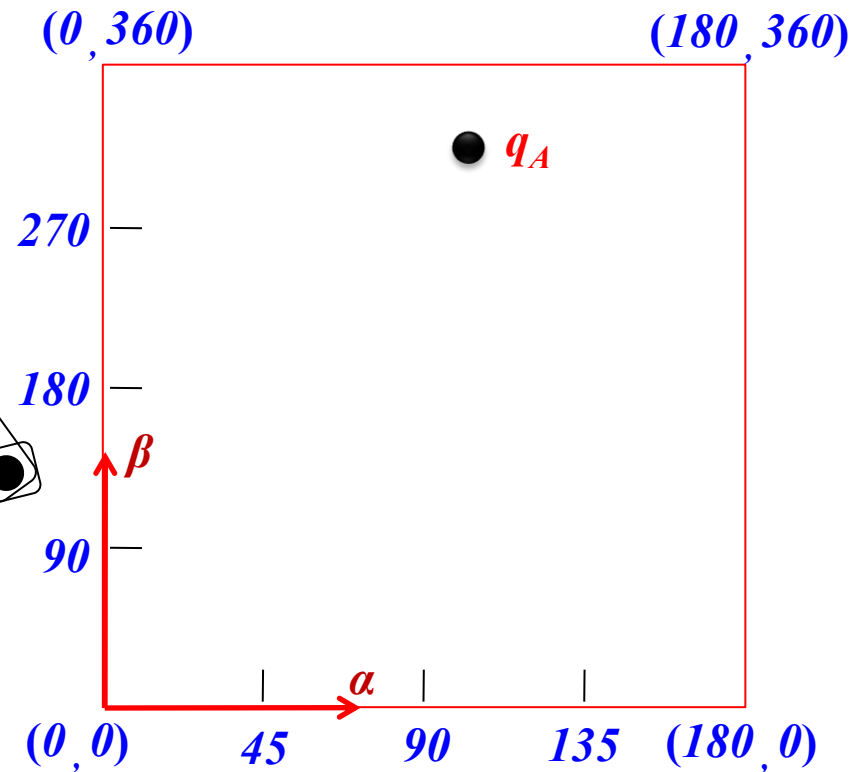
- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples

Some More Food for Thought



An obstacle in the robot's workspace?

Where can we put $\bullet q_B$?



Torus

(wraps horizontally and vertically)



Defining futures

Configuration Space



- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples
 - Obstacle in C-Space

Obstacles in C-Space

- Let q denote a point in a configuration space Q
- The path planning problem is to find a mapping $c:[0,1] \rightarrow Q$ s.t. no configuration along the path intersects an obstacle
- Recall a workspace obstacle is WO_i
- A *configuration space obstacle* QO_i is the *set of configurations* q at which the robot intersects WO_i , that is
 - $QO_i = \{q \in Q \mid R(q) \cap WO_i \neq \emptyset\}$
- The *free configuration space* (or just *free space*) Q_{free} is
$$Q_{free} = Q \setminus (\cup QO_i)$$
- The free space is generally an open set
 - A *free path* is a mapping $c:[0,1] \rightarrow Q_{free}$
 - A *semi-free path* is a mapping $c:[0,1] \rightarrow cl(Q_{free})$



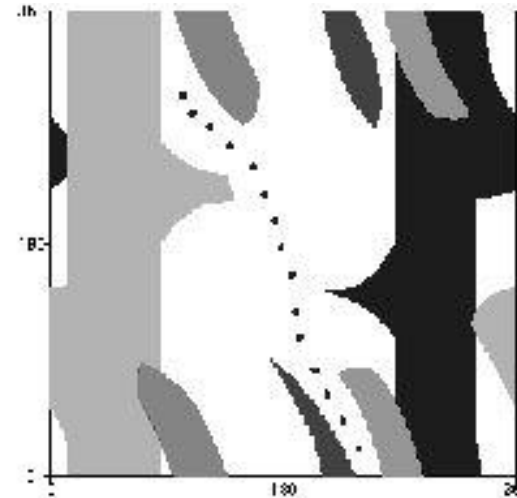
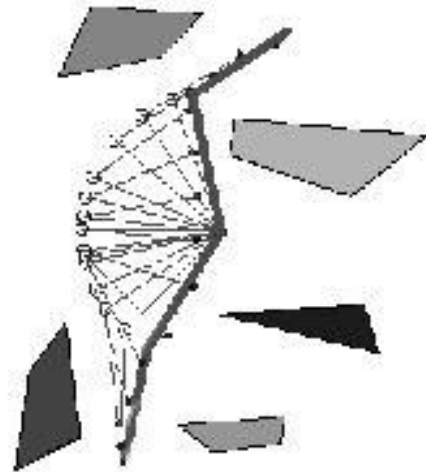
Configuration Space



- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples
 - Obstacle in C-Space

Two Joint Planar Arm

- When a robot is slightly more complex it becomes extremely difficult to compute the configuration space, therefore we use a grid based approach for complex objects
- For a two joint planar arm ($Q = T^2$) we can define a grid on the surface and for each point on the grid we can test to see if a certain configuration of the robot will collide with an obstacle.
- If we let each grid point be represented by a pixel we can visualize the configuration space by colouring the pixels appropriately



Defining futures

Configuration Space



- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples
 - Obstacle in C-Space
 - C-Space Dimensions

Dimensions of a Configuration Space

- The *dimension* = *number of parameters* necessary to uniquely specify configuration (= DOF)
- *One-way* – Explicitly generate a parameterization (e.g., two-bar linkage)



Defining futures

Configuration Space



- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples
 - Obstacle in C-Space
 - C-Space Dimensions

Dimensions of a Configuration Space

- *Another way* – Start with many parameters and add (indept) constraints
 - Suppose that a planar rigid robot is free to translate and rotate in a plane
 - Define A, B, C to be three distinct points on the body
 - We are free to choose the position of A (coordinates (x_A, y_A) in a plane)
 - Now choose the coordinates of $B (x_B, y_B)$ however, rigidity requires
$$d(A, B) = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$
This constrains B to lie on a circle of radius $A (a, b)$ centred at (x_A, y_A) our only freedom is in choosing θ
 - Now choose coordinates of $C (x_C, y_C)$ this choice has 2 constraints
$$d(A, C) = \sqrt{(x_A - x_C)^2 + (y_A - y_C)^2} \quad \text{and} \quad d(B, C) = \sqrt{(x_B - x_C)^2 + (y_B - y_C)^2}$$
 - In fact C is already chosen for us. Every point on the body is chosen by selecting (x_A, y_A, θ) so the robot has 3 DOF and its space is $\mathbb{R}^2 \times S^1$



Defining futures



Configuration Space

- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples
 - Obstacle in C-Space
 - C-Space Dimensions

Dimensions of a Configuration Space

Constraints – Two Joints Planar Arm

- *The other way*
 - Start with many parameters (*12 coordinates* since each arm has *6 DOF*) and add (independent) constraints
 - Each link can be represented as a rigid spatial body (*6 DOF*)
 - Three constraints for each link restrict the links to the plane???
 - Two constraints restrict a point on the first link to be at a fixed location in the plane???
 - Once angle of first link is chosen two constraints restrict a point on the first link to be at a fixed location

$$(12) \text{ Coordinates} - (2 \times 5) \text{ Constraints} = 2 \text{ DOF}$$



Configuration Space



- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples
 - Obstacle in C-Space
 - C-Space Dimensions
 - Constraints

Constraints

- The distance constraints for a translating and rotating object in a 2D space are examples of a *Holonomic* constraint.
- *Holonomic* Constraint:
 - Purely a function of configuration variables (and time), i.e., of the form:
 $g(q, t) = 0$
 - Each linearly independent *Holonomic constraint* reduces the dimension of the configuration space by 1
 - A system defined by n coordinates subject to m constraints has $(n - m)$ dimensions
- *Non-Holonomic* Constraint:
 - Includes velocity constraints (derivatives of configuration), i.e., of the form:
 $g(q, \dot{q}, t) = 0$
 - Do not reduce the dimension of the configuration space



Defining futures

Configuration Space



- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples
 - Obstacle in C-Space
 - C-Space Dimensions
 - Constraints
 - + Non-Holonomic

Non-Holonomic Constraints

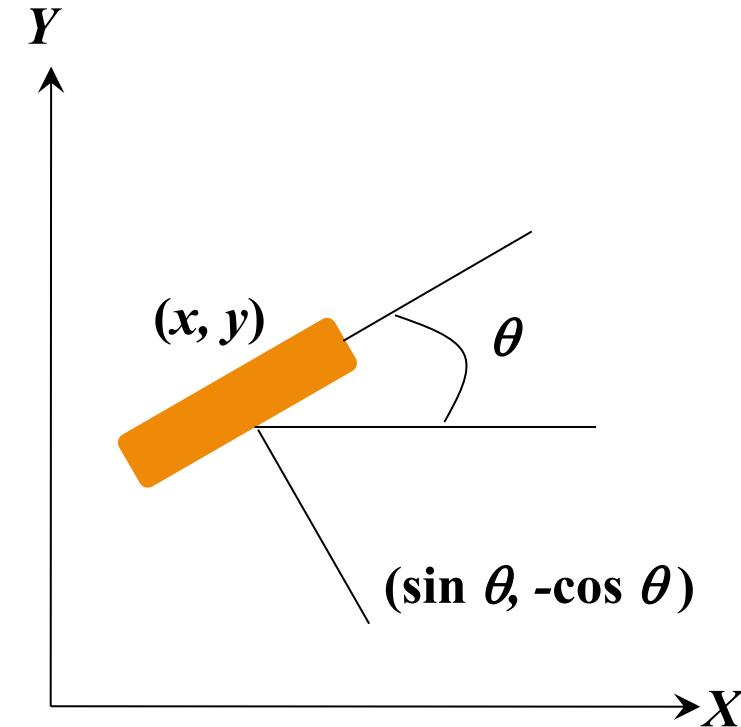
Example : Unicycle

- A unicycle cannot move sideways

- Let

$$\dot{q} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix}$$

- and let $n = (\sin \theta, -\cos \theta, 0)$
- In this case constraint is $n \dot{q} = 0$



Configuration Space



- C- Space
 - Non-Point Robots
 - What's a C-Space
 - Examples
 - Obstacle in C-Space
 - C-Space Dimensions
 - Constraints
 - + Non-Holonomic

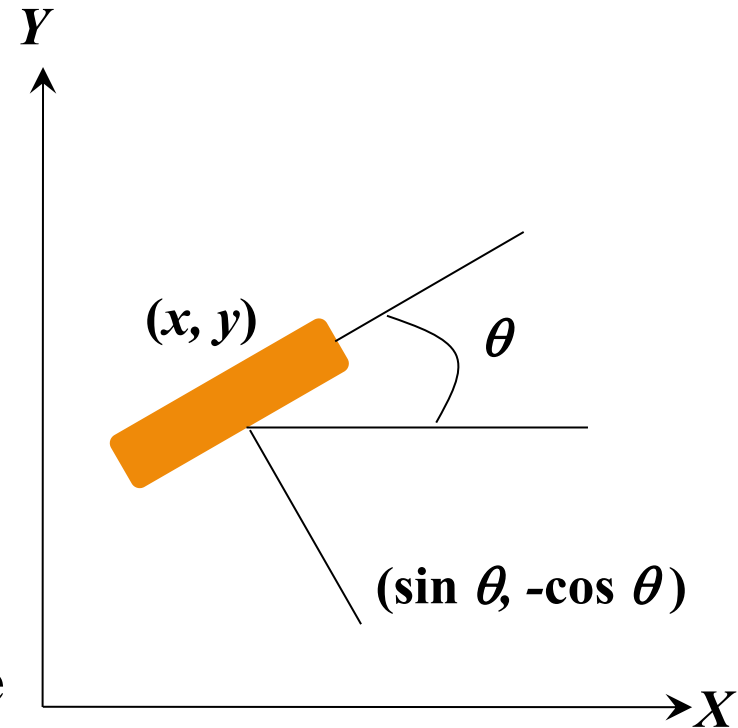
Non-Holonomic Constraints

Example : Unicycle

- The unicycle can move in two different directions expressed by

$$g_1(q) = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \text{ and } g_2(q) = \begin{bmatrix} \cos \theta \\ \sin \theta \\ 0 \end{bmatrix}$$

- Note that the unicycle motion is
$$\dot{q} = u_1 g_1(q) + u_2 g_2(q)$$
where u_1 and u_2 are the controls
- How many DOFs does the unicycle have?



Non-Holonomic constraints do NOT reduce dimensions of the C-space

